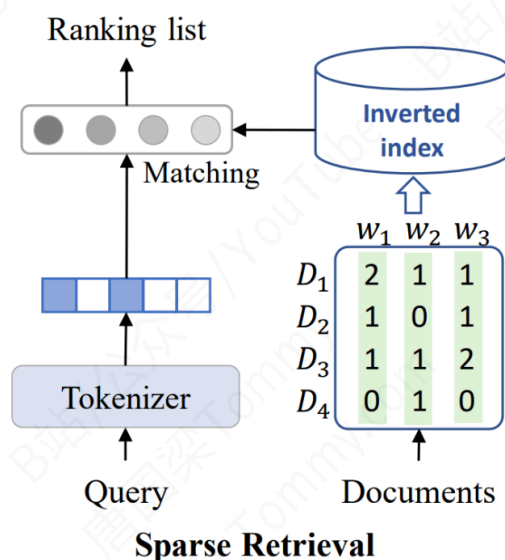


在RAG框架中，"R"代表检索。检索器的核心作用是快速而准确地从庞大的知识库中找到与用户查询最相关的信息。

2.1 稀疏检索 SR, Sparse Retriever

稀疏检索器主要基于文档和查询的词项匹配，是一种传统但强大的信息检索技术。

- **BM25**：作为稀疏检索的标准算法，BM25通过考虑词项频率（TF）和逆文档频率（IDF）来评估查询和文档之间的相关性。由于其简单有效，BM25常被用作稀疏检索的基线模型。
- **rank-bm25**：这是BM25的一个基础实现，没有包含复杂的预处理步骤，因其易于使用，在Python社区中被广泛集成，如LangChain和Llama-index等库。
- **Pyserini BM25**：Pyserini提供了一种更高级的BM25实现，包含了词干提取和去除特定语言停用词等预处理步骤，旨在提高检索的精确度和效率。



2.1.1 BM25

BM25是一种广泛使用的排名函数，用于信息检索中以评估文档相对于给定搜索查询的相关性。它是Okapi BM25算法的变体，基于概率检索框架。BM25考虑了词频（term frequency, TF）和逆文档频率（inverse document frequency, IDF），以及文档长度对搜索查询中每个词的重要性的影响。下面是BM25算法的详细解释及其Python实现步骤：

(1) BM25算法原理

BM25的评分函数基于以下几个关键概念：

- **TF (Term Frequency)**: 词项在文档中出现的频率。BM25对传统的TF值进行了调整，以防止长文档过度放大词项的影响。
- **IDF (Inverse Document Frequency)**: 词项的普遍重要性的度量。IDF值越高，表示词项在较少的文档中出现，因此具有更高的区分能力。
- **文档长度归一化**: BM25通过考虑文档长度与平均文档长度的比例来调整评分，以公平对待长文档和短文档。

BM25的评分函数为每个文档计算一个分数，表示该文档与查询的相关性。分数越高，文档与查询的相关性越高。BM25的评分公式如下：

$$\text{Score}(D, Q) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot (1 - b + b \cdot \frac{|D|}{\text{avgdl}})}$$

其中：

- D 是文档
- Q 是查询，包含词项 q_i
- $f(q_i, D)$ 是词项 q_i 在文档 D 中的频率
- $|D|$ 是文档 D 的长度
- avgdl 是文档集中文档的平均长度
- $\text{IDF}(q_i)$ 是词项 q_i 的逆文档频率
- k_1 和 b 是自由参数，通常 $k_1 \in [1.2, 2.0]$ 和 $b = 0.75$

2.1.2 SR的特点

(1) 优势

1. **高效性**：稀疏检索器因其简单的匹配机制和优化过的索引结构而具有高效的检索速度，尤其适用于大规模文档集合。
2. **解释性**：由于稀疏检索依赖于词项的显式匹配，它的检索结果通常更易于解释。用户和开发者可以直观地理解为何某个文档与查询相关。
3. **简单性**：稀疏检索器的实现相对简单，不需要复杂的模型训练过程，易于部署和维护。
4. **鲁棒性**：在处理包含关键词或具有明确查询意图的搜索时，稀疏检索器能够提供可靠的性能。

(2) 劣势

1. **语义匹配限制**：稀疏检索器在处理语义上相关但不共享显式词项的查询和文档时性能受限，可能错过语义上相关但词项不匹配的文档。
2. **对长尾查询的处理**：对于非常具体或罕见的查询，稀疏检索器可能因为缺乏直接的词项匹配而难以检索到高度相关的文档。
3. **更新和维护开销**：虽然稀疏检索器在建立索引时效率较高，但对索引的更新（如添加新文档）可能需要额外的处理，尤其是在文档集合频繁变化的情况下。
4. **查询依赖性**：稀疏检索器的性能高度依赖于查询的质量，对于模糊或多义性查询，可能无法准确理解用户的真实意图。

>>> 举例讲解

BM25案例实践.ipynb